UNIVERSIDAD ESAN · Robótica · Sección S-003

Proyecto CapyTown · Grupo 4

Informe RC-0: primer arranque y calibración de la odometría del Yahboom MicroROS-Pi5

Rol	Integrante	Responsabilidad en RC-0
Integración	Simón Contreras	Bringup, ROS_DOMAIN_ID, rosbag2, bags, repositorio y tag
Control	Aaron Cruzado	Calibración de r y b_eff; wheel_params.yaml
Planificación	Yoel Miranda	scripts/rc0_pruebas.sh: secuencia de pruebas
Percepción	Joaquín Rodríguez	plot_odom.py: lectura de bags y gráficas

Repositorio: github.com/YoelMiranda/capytown_G4 (tag v1.0-rc0) · Fecha de ejecución: 2 de noviembre de 2026

Resumen. Se puso en marcha el robot con ROS 2 Humble en un contenedor persistente y se calibró la odometría con un corrector propio (*odom_corrector*) que reescala la odometría del firmware. El radio efectivo de rueda quedó en **r = 0.03056 m** (6.0 % menor que el valor del firmware) y la separación efectiva en **b_eff = 0.2176 m** (9.3 % menor). Con estos valores, la recta de verificación tuvo un error de 0.54 % y el giro de convergencia un error de 0.14 %. En la validación sin recalibrar (variante A2), las rectas dieron 1.99 % y 1.24 %, mientras que el giro horario dio 4.00 %. Este último resultado apunta a una asimetría entre los dos sentidos de giro que un único b_eff no puede corregir.

1. Objetivo

El RC-0 busca dejar el robot operativo y corregir la odometría que publica su firmware, de modo que la distancia y el ángulo estimados coincidan con lo que se mide sobre la pista. Para ello se calibran dos parámetros del modelo de tracción diferencial: el radio efectivo de rueda *r*, con una recta de 1.0 m, y la separación efectiva entre ruedas *b_eff*, con un giro en sitio de 360°. Por último, se valida el resultado en otra variante de la pista, en sentido contrario y sin recalibrar.

2. Sistema y arquitectura

2.1 Plataforma

El robot usa una Raspberry Pi 5 con Raspberry Pi OS y una placa ESP32 con firmware micro-ROS (nodo */YB_Car_Node*). La placa publica `/odom_raw`, `/imu`, `/scan` y `/battery`, y escucha `/cmd_vel`. ROS 2 Humble no está instalado en el sistema base, sino en el contenedor Docker *ros2_dev* (imagen yahboomtechnology/ros-humble:4.1.2). Este contenedor es persistente, porque se reinicia solo con el robot, y monta el workspace desde el host (`/home/pi/yahboomcar_ws` → `/root/yahboomcar_ws`). Por ello, el código sobrevive aunque el contenedor se recree. El agente micro-ROS corre en un contenedor aparte (*micro_ros_agent*). Todo el sistema trabaja con **ROS_DOMAIN_ID = 20**.

2.2 Corrección de la odometría

El nodo *odom_corrector*, lanzado con `bringup.launch.py`, toma `/odom_raw` y publica `/odom` y la transformada odom → base_footprint. Para ello reescala cada incremento de avance y de giro:

$$\Delta s' = k_lin \cdot \Delta s, \quad k_lin = r / r0$$
$$\Delta\theta' = k_ang \cdot \Delta\theta, \quad k_ang = (r / r0) \cdot (b0 / b_eff)$$

Aquí r0 = 0.0325 m y b0 = 0.240 m son los valores que asume el firmware, y no se modifican. Es decir, la calibración cambia la *estimación* de la pose, pero no la forma en que se mueve el robot. Los parámetros se leen de `config/wheel_params.yaml` y también se pueden cambiar en caliente con `ros2 param set`, lo que se

aprovechó en las iteraciones de la Parte C. Con los valores finales, k_lin = 0.9403 y k_ang = 1.0371. Ambos factores se comprobaron en los bags: el cociente entre `/odom` y `/odom_raw` coincide con ellos en todas las corridas.

3. Parte A: arranque y verificación

Con el agente micro-ROS activo y el bringup lanzado, se ejecutó `rc0_pruebas.sh check`, que guarda en `setup_log.txt` la lista de nodos, la de tópicos y la frecuencia de los tópicos principales:

Tabla 1. Verificación del arranque (setup_log.txt)

Elemento	Resultado	Comentario
Nodos	/YB_Car_Node, /odom_corrector	Firmware y corrector, un solo corrector activo
Tópicos	/cmd_vel, /odom, /odom_raw, /odom_reset, /tf, /imu, /scan, /battery, …	Todos los requeridos presentes
/odom	≈ 11.1 Hz	Misma tasa que /odom_raw
/odom_raw	≈ 11.1 Hz	Publicado por la ESP32
/imu	≈ 22 Hz	
rosbag2	Preinstalado, almacenamiento sqlite3	No fue necesario el Anexo A

La imagen de Yahboom ya trae rosbag2, compilado desde el código fuente (por eso no aparece en `dpkg`), con el almacenamiento sqlite3. La opción `--list-storage` del enunciado no existe en esta versión, así que el funcionamiento se comprobó con una grabación corta de `/odom_raw`.

Nota sobre las fechas. La Raspberry Pi 5 no tiene batería de reloj y, conectada solo por su hotspot, tampoco tiene internet. Por eso las marcas de tiempo de `setup_log.txt`, de `calibration_log.csv` y de los bags muestran el reloj interno del robot (15 de julio de 2026). Todas las pruebas se hicieron el 2 de noviembre de 2026.

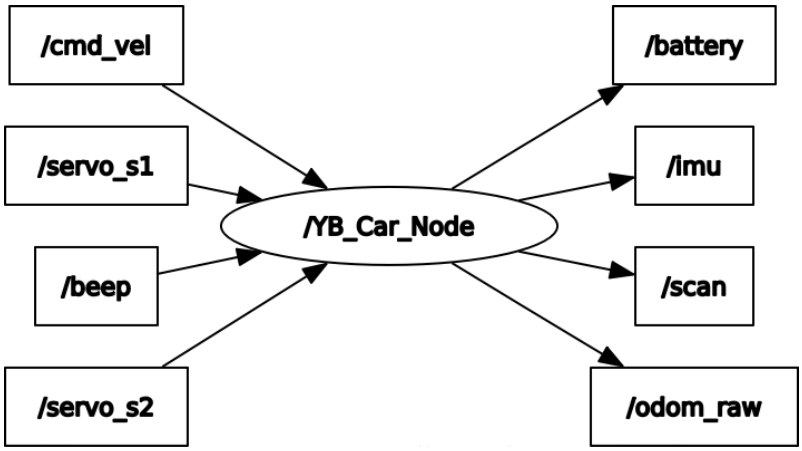


Figura 1. Captura de rqt_graph (rqt_graph.png): el nodo del firmware y sus tópicos. En esta captura no aparece /odom_corrector; su presencia y la de /odom, /tf y /odom_reset quedan registradas en setup_log.txt.

4. Metodología de pruebas

Todas las pruebas se lanzaron con `scripts/rc0_pruebas.sh`. En cada corrida, el script resetea la pose de `/odom`, graba un bag con `/odom /odom_raw /cmd_vel /tf` y, al terminar, imprime un resumen con `plot_odom`. En la recta comanda 0.10 m/s durante 10 s (100 mensajes a 10 Hz). En el giro llama a `calibrate_beff`, que gira a ±0.5 rad/s hasta que `/odom` acumula 360° y luego pide el ángulo físico medido.

Tabla 2. Variantes asignadas a la sección S-003

Uso	Variante	Recta de 1.0 m	Giro en sitio
Calibración (B y C)	A1 «Torre»	B4 → C4, de oeste a este	tile B2, antihorario (+0.5 rad/s)
Validación (D)	A2 «Escalón»	C4 → D4, de este a oeste	tile A1, horario (−0.5 rad/s)

La distancia real (d_real) se midió con cinta métrica, desde la línea de inicio hasta el eje de las ruedas. El giro real se midió con transportador: se pegó una cinta en el piso alineada con el robot y, al final, se registró 360° ± δ según si el robot se pasó o se quedó corto. El error se define como |odom − real| / real · 100. Las reglas de calibración son:

$$r_nuevo = r \cdot d_real / d_odom$$
$$b_eff_nuevo = b_eff \cdot \theta_objetivo / \theta_medido$$

5. Parte B: recta de 1.0 m y calibración de r

Con el valor del firmware (r = 0.0325 m) se hicieron tres corridas en A1. En las tres, la odometría marcó más distancia que la real, con errores entre 5.7 % y 7.2 %, muy por encima del umbral de 2 %. Por ello se corrigió r con el promedio de las tres razones d_real / d_odom:

$$r_nuevo = 0.0325 \times 0.94037 = 0.03056\ m$$

Tabla 3. linea_recta.csv (desvío lateral según la odometría)

Corrida	r [m]	d_real [m]	d_odom [m]	Error [%]	Desvío lateral [m]	Bag
1	0.0325	1.030	1.1044	7.22	0.0062	rc0_recta
2	0.0325	1.022	1.0841	6.08	0.0363	rc0_recta_2
3	0.0325	1.025	1.0838	5.74	−0.0070	rc0_recta_3
4 (verificación)	0.03056	1.152	1.1458	**0.54**	−0.0139	rc0_recta_4

La corrida de verificación, ya con r = 0.03056 m, dio 0.54 %. Además, como los bags guardan `/odom_raw`, se pudo recalcular qué error habrían tenido las corridas 1 a 3 con el r final: 0.83 %, 0.26 % y 0.57 %. Es decir, el valor calibrado funciona sobre todas las corridas, no solo sobre la de verificación (Figura 2).

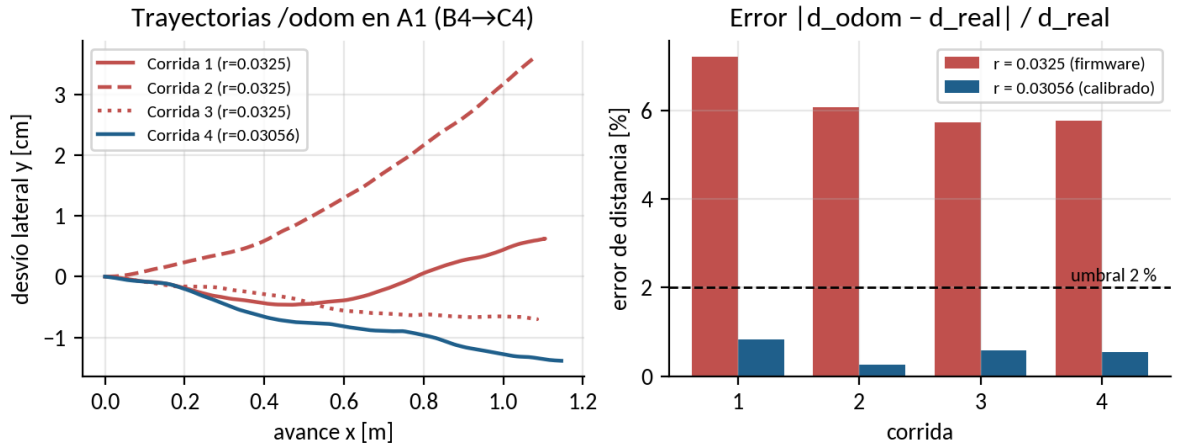


Figura 2. Izquierda: trayectorias de /odom en A1. Derecha: error de distancia de cada corrida con el r del firmware y con el r calibrado (recalculado a partir de /odom_raw).

Un detalle importante es que el mismo comando de 1.0 m produjo recorridos reales de 1.022 a 1.152 m. El firmware no controla la distancia recorrida: aplica una velocidad durante un tiempo, y la inercia al frenar, el piso o la batería cambian el resultado. Esto no afecta la calibración, que compara la odometría con la distancia

real de cada corrida, pero sí importa para el RC-1 (sección 9).

6. Parte C: giro de 360° y calibración de b_eff

Con r ya calibrado, se iteró el giro antihorario en el tile B2. En cada iteración, `calibrate_beff` calcula un b_eff nuevo, que se aplicó en caliente con `ros2 param set /odom_corrector wheel_separation` antes del siguiente giro. La Tabla 4 reproduce `calibration_log.csv`.

Tabla 4. calibration_log.csv (/odom = ángulo al detener el giro)

Giro	b_eff antes [m]	/odom [°]	Real [°]	Error [%]	b_eff sugerido [m]	Bag
it1	0.2400	360.2	377.0	4.72	0.2292	rc0_giro_it1
it2	0.2292	360.7	369.0	2.50	0.2236	rc0_giro_it2_2
it3	0.2236	360.8	370.0	2.78	0.2176	rc0_giro_it3
it4a	0.2176	360.5	368.0	2.22	0.2129	rc0_giro_it4
it4b	0.2176	361.4	360.5	**0.14**	0.2173	rc0_giro_it4_2
oficial	0.2176	361.9	361.0	**0.28**	0.2170	rc0_giro
repetición	0.2176	361.5	351.0	2.50	0.2232	rc0_giro_2

Se adoptó **b_eff = 0.2176 m**, el valor con el que la iteración it4b convergió (0.14 %) y con el que se grabó el giro oficial (0.28 %). No se adoptó el 0.2173 sugerido en it4b, porque esa diferencia (0.1 %) es menor que la resolución del transportador. Antes del registro se descartaron dos lecturas: una que repetía 377° con b_eff = 0.2292 y otra de 290° con b_eff = 0.2236. Ambas eran incompatibles con el cambio de b_eff aplicado (un cambio de 2–5 % en b_eff solo puede mover el giro real unos pocos grados), por lo que esos giros se repitieron.

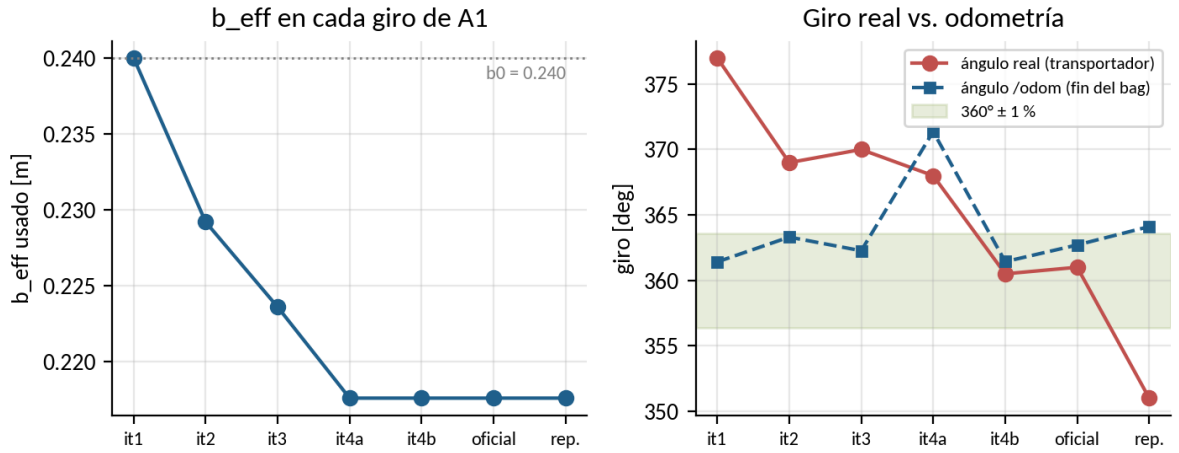


Figura 3. Izquierda: b_eff usado en cada giro. Derecha: ángulo real medido y ángulo de /odom al final del bag; la banda verde es 360° ± 1 %. En it4a el bag registra unos 11° más después de la parada, posiblemente por mover el robot durante la medición.

Con el mismo b_eff = 0.2176 m hay cuatro mediciones: 368.0°, 360.5°, 361.0° y 351.0°. Su promedio es 360.1° y su desviación estándar 7.0°. El promedio coincide con la odometría (361.3° en promedio, una diferencia de 0.3 %), así que la calibración es correcta en media. En cambio, la dispersión de cada giro individual (cerca de ±2 %) es mayor que el umbral de 1 %. Esa dispersión resume dos fuentes: la lectura manual del transportador y el deslizamiento variable de las ruedas al girar.

7. Parte D: validación en A2 sin recalibrar

Con r = 0.03056 m y b_eff = 0.2176 m fijos, se repitieron las pruebas en la variante A2: dos rectas de este a oeste y un giro en sentido horario. El giro se registró directamente con `calibrate_beff` en modo validación.

Tabla 5. validacion.csv

Prueba	Real	Odometría	Error [%]	Bag
Recta 1	1.045 m	1.0242 m	1.99	rc0_val_recta
Recta 2	1.255 m	1.2395 m	1.24	rc0_val_recta_2
Giro 360° horario	376.0°	360.9°	4.00	rc0_val_giro

Las dos rectas quedan por debajo del 2 %, aunque el robot recorrió distancias muy distintas (1.045 y 1.255 m) con el mismo comando. La odometría siguió bien ambos recorridos, que es lo que se busca validar. A diferencia de la Parte B, aquí la odometría marcó algo *menos* que la cinta; con solo dos corridas no es posible separar si se debe al cambio de sentido o a la dispersión normal entre corridas (las razones d_real / d_odom_raw de las seis rectas varían ±0.9 %).

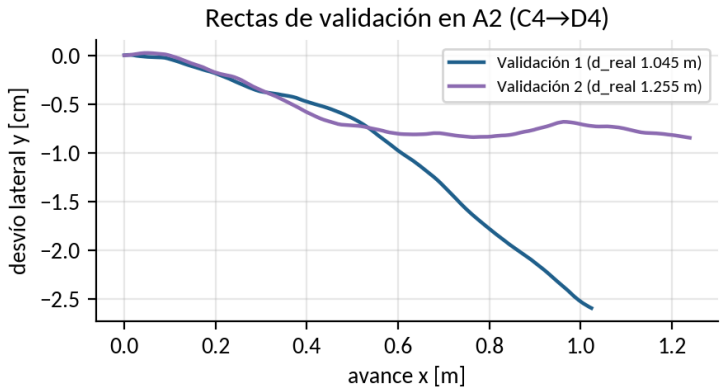


Figura 4. Trayectorias de /odom en las rectas de validación (A2).

El giro horario, en cambio, tuvo un error de 4.00 %: el robot giró 376° mientras la odometría marcaba 360.9°. Comparado con la odometría del firmware, en sentido horario el robot giró 376 / 350.2 = 1.074 veces lo que estimó la ESP32, mientras que en el giro oficial antihorario esa razón fue 361 / 349.7 = 1.032. Esta diferencia de cerca de 4 % sugiere una asimetría entre los dos sentidos de giro (motores, control de la placa o roce de las ruedas), que un modelo con un solo b_eff no puede corregir. Como referencia, el b_eff que ajustaría el giro horario sería cercano a 0.210 m. Al tratarse de una sola medición, se recomienda repetirla antes de sacar una conclusión firme.

Bonus. No se intentó. Con la asimetría observada, un giro horario de 720° acumularía un error cercano a 29°, muy por encima del límite de 5°, y ninguna recta de A2 alcanzó el 1 % (la mejor fue 1.24 %).

8. Parte E: gráficas y resumen

`plot_odom` generó, para cada bag, la trayectoria x-y y la orientación θ(t) en `plots/`, y el resumen de las corridas oficiales en `plots/resumen.csv` (Tabla 6). También se graficaron `rc0_recta_4` y `rc0_giro` con `--topic /odom_raw` (`plots/raw/`) para comparar la odometría sin corregir.

Tabla 6. plots/resumen.csv (tópico /odom)

Bag	Tipo	Duración [s]	Mensajes	d_odom [m]	Desvío [m]	Giro [°]
rc0_recta	recta	16.56	185	1.1044	0.0062	0.53
rc0_recta_4	recta	18.57	207	1.1458	−0.0139	−0.34
rc0_giro	giro	35.77	389	0.0031	0.0002	362.70
rc0_val_recta	recta	17.14	192	1.0242	−0.0260	−1.35
rc0_val_recta_2	recta	17.45	195	1.2395	−0.0085	−0.58
rc0_val_giro	giro	43.93	489	0.0049	−0.0036	−363.15

Tabla 7. Odometría sin corregir frente a la corregida (valores al final del bag)

Bag	/odom_raw (firmware)	/odom (corregida)	Medición real	Error raw → corregido
rc0_recta_4	1.2185 m	1.1458 m	1.152 m	5.77 % → 0.54 %
rc0_giro	349.7°	362.7°	361.0°	3.13 % → 0.47 %

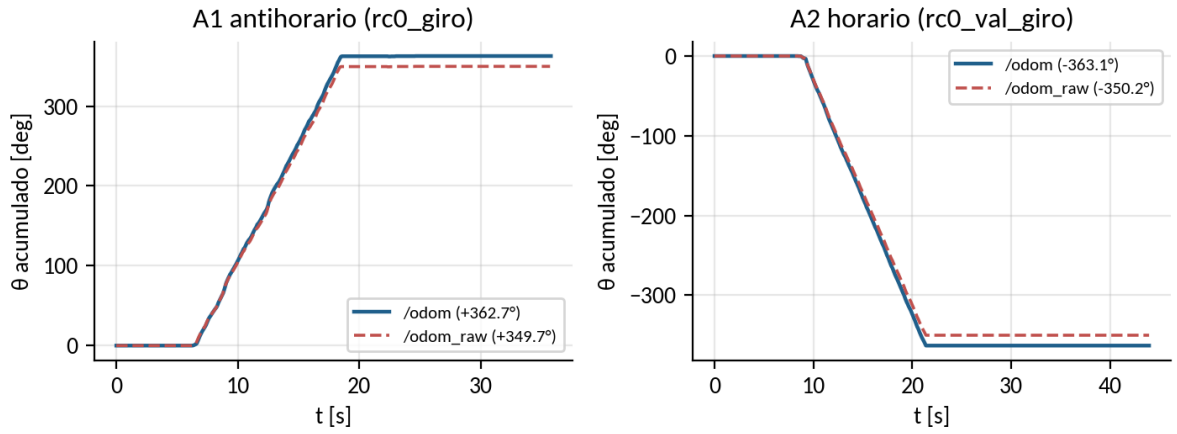


Figura 5. θ(t) de /odom y /odom_raw en el giro oficial antihorario (A1) y en el giro de validación horario (A2).

9. Discusión

- **Magnitud de la corrección.** El firmware sobrestima la distancia en cerca de 6 % y subestima el giro en cerca de 3.6 % (k_lin = 0.940, k_ang = 1.037). La corrección reduce el error de la recta de 5.8 % a 0.5 % y el del giro de 3.1 % a 0.5 %.
- **Incertidumbre de medición.** La cinta métrica tiene una resolución cercana a 0.5 cm (0.5 % en 1 m), suficiente para el umbral de 2 %. El transportador, en cambio, se leyó con una variación de unos ±5°, comparable al umbral de 1 % (3.6°). Por eso la Parte C requirió más iteraciones y se descartaron dos lecturas.
- **Movimiento en lazo abierto.** El mismo comando produjo rectas de 1.022 a 1.255 m y giros reales de 351° a 377°. La odometría corregida describe bien lo que hizo el robot, pero el robot no hace siempre lo mismo. Para el RC-1 (cuadrado con `run_square`), conviene cerrar el lazo con `/odom` en lugar de comandar por tiempo.
- **Asimetría de giro.** El resultado de validación en sentido horario indica que el factor de giro depende del sentido. Si se confirma, una opción es usar un b_eff distinto para cada sentido en `odom_corrector`.

10. Problemas encontrados y soluciones

Tabla 8. Incidencias durante el RC-0

Problema	Causa	Solución
Se borraron los archivos del robot	El código vivía dentro de un contenedor efímero	Se clonó el repo en ~/yahboomcar_ws/src, montado como volumen en el contenedor persistente ros2_dev
El bag de la recta no se cerraba y el script quedaba colgado	ros2 bag record lee el teclado; lanzado en segundo plano, el sistema lo detiene (SIGTTIN) y no atiende el SIGINT	Se lanzó el recorder con < /dev/null (corregido en scripts/rc0_pruebas.sh)
El bringup seguía con r = 0.0325 tras editar el YAML	wheel_params.yaml se instala como copia en install/	colcon build tras cada edición; en las iteraciones, ros2 param set
Dos nodos /odom_corrector	Cerrar una terminal no detiene el proceso dentro del contenedor	pkill -f odom_corrector y verificación con ros2 node list
Una recta sin /odom en el bag	El bringup no estaba corriendo	Se descartó la corrida y se repitió
ros2 bag record --list-storage falla	La opción no existe en esta versión de rosbag2	Se verificó con una grabación de prueba (sqlite3)
Tracebacks en setup_log.txt	timeout corta ros2 topic hz con SIGTERM	Las frecuencias son válidas; con timeout -s INT la salida queda limpia

11. Conclusiones

- El robot quedó operativo con ROS 2 Humble en un contenedor persistente, con el workspace en un volumen del host, y la odometría corregida se publica a 11 Hz junto con la transformada odom → base_footprint.
- La calibración dio **r = 0.03056 m** y **b_eff = 0.2176 m**. Con ellos, el error fue de 0.54 % en la recta (umbral 2 %) y de 0.14 % en el giro de convergencia (umbral 1 %).
- En la validación en A2, las rectas se mantuvieron bajo el 2 % (1.99 % y 1.24 %). El giro horario dio 4.00 %, lo que indica una asimetría entre sentidos de giro que conviene confirmar con más corridas.
- La principal limitación fue la medición manual del ángulo. Para próximos retos, una referencia más precisa (por ejemplo, una marca larga en el piso o el IMU como verificación cruzada) reduciría la dispersión.

Anexo A. Reproducción

Dentro del contenedor (`./ros2_humble.sh`) y con ROS_DOMAIN_ID = 20:

```
# Terminal 1: bringup
cd ~/yahboomcar_ws && colcon build --symlink-install --packages-select capytown_esan
source install/setup.bash && ros2 launch capytown_esan bringup.launch.py

# Terminal 2: pruebas, desde la raíz del repo
bash scripts/rc0_pruebas.sh check
bash scripts/rc0_pruebas.sh recta A1 rc0_recta
bash scripts/rc0_pruebas.sh giro A1 calibracion 360 rc0_giro_it1
bash scripts/rc0_pruebas.sh giro A1
bash scripts/rc0_pruebas.sh recta A2 rc0_val_recta
bash scripts/rc0_pruebas.sh giro A2
ros2 run capytown_esan plot_odom bags/rc0_recta bags/rc0_recta_4 bags/rc0_giro \
  bags/rc0_val_recta bags/rc0_val_recta_2 bags/rc0_val_giro --out plots --resumen
plots/resumen.csv
```

Las pruebas unitarias de `bag_utils` (`python3 -m pytest test/test_bag_utils.py`) pasan: 8 de 8.

Anexo B. Evidencias en el repositorio

Tabla 9. Archivos de evidencia

Archivo	Contenido
setup_log.txt, rqt_graph.png	Parte A: nodos, tópicos, frecuencias y grafo
linea_recta.csv, bags/rc0_recta*	Parte B: cuatro rectas en A1
calibration_log.csv, bags/rc0_giro*	Parte C: iteraciones, giro oficial y repetición
validacion.csv, bags/rc0_val_*	Parte D: rectas y giro en A2
plots/, plots/raw/, plots/resumen.csv	Parte E: gráficas x-y y θ(t), resumen y comparación con /odom_raw
config/wheel_params.yaml	r = 0.03056 m, b_eff = 0.2176 m